Tytuł projektu

Implementacja rozmycia Gaussa z optymalizacją wektorową AVX2

Opis projektu

Mój projekt asemblerowy zakładał stworzenie bibliotek DLL w języku asemblera oraz C++ których zadaniem byłoby pobieranie obrazka w formacie RGB (24-bitowa bitmapa) i przetwarzanie jej przez zastosowanie filtru rozmycia Gaussa z wykorzystaniem 2D separowalnego jądra konwolucji 3×3. Filtr zaimplementowano przy użyciu arytmetyki stałoprzecinkowej Q7 (fixed-point), co eliminuje potrzebę operacji zmiennoprzecinkowych i znacząco przyspiesza obliczenia. Utworzyłem również prostą aplikację okienkową w języku wysokiego poziomu (C# WinForms) której zadaniem jest ukazanie działania programu oraz umożliwienie porównania wydajności obu implementacji przy różnej liczbie wątków roboczych.

Parametrami wejściowymi obu bibliotek DLL są:

- inputImage / tempBuffer – wskaźnik na tablicę bajtów zawierającą dane bitmapy RGB
- outputImage / tempBuffer – wskaźnik na bufor wyjściowy
- width – szerokość obrazu w pikselach
- stride – rzeczywista szerokość wiersza w bajtach (z wyrównaniem)
- startY, endY – zakres wierszy do przetworzenia (podział na wątki)
- height – wysokość obrazu (tylko w fazie pionowej)

Algorytm rozmycia Gaussa został zdekomponowany na dwie fazy (separowalna konwolucja 2D):

1. **Konwolucja pozioma (horizontal blur)** – przetwarzanie każdego wiersza obrazu z jądrem 1D: $[0.125, 0.75, 0.125]$
2. **Konwolucja pionowa (vertical blur)** – przetwarzanie każdej kolumny z tym samym jądrem 1D

Działanie programu ASM

Reprezentacja Q7 Fixed-Point

Współczynniki jądra Gaussa zostały zakodowane w arytmetyce stałoprzecinkowej Q7 (7 bitów ułamkowych):

- $K_0 = 0x10 = 16_{10} \rightarrow 16/128 = 0.125$
- $K_1 = 0x60 = 96_{10} \rightarrow 96/128 = 0.75$
- $K_2 = 0x20 = 32_{10} \rightarrow 32/128 = 0.25$ (dla pikseli brzegowych)

Faza 1: Konwolucja pozioma (GaussianBlurASM_Horizontal.asm)

Algorytm wykorzystuje 256-bitowe rejestry AVX2 (ymm0–ymm7) do równoległego przetwarzania 16 kanałów kolorów (RGB) jednocześnie, co odpowiada około 5 pikselom na iterację.

Kluczowe kroki algorytmu:

1. **Przygotowanie wektorów jądra:**

```
mov eax, 10h
vmovq xmm0, rax
vpbroadcastw ymm3, xmm0 ; ymm3 = [0x10, 0x10, 0x10, ...]
```

```
mov eax, 60h
vmovd xmm0, eax
vpbroadcastw ymm4, xmm0 ; ymm4 = [0x60, 0x60, 0x60, ...]

mov eax, 20h
vmovd xmm0, eax
vpbroadcastw ymm6, xmm0 ; ymm6 = [0x20, 0x20, 0x20, ...]
```

2. **Wczytanie pikseli i rozpakowanie 8-bit → 16-bit:**

   ```
   vpmovzxbw ymm0, xmmword ptr [rsi+rax] ; p[x-1]
   vpmovzxbw ymm1, xmmword ptr [rsi+rax+3h] ; p[x]
   vpmovzxbw ymm2, xmmword ptr [rsi+rax+6h] ; p[x+1]
   ```

   Instrukcja vpmovzxbw (packed move with zero-extend byte to word) rozszerza 16 bajtów 8-bitowych na 16 wartości 16-bitowych w rejestrze YMM, wypełniając starsze bity zerami.

3. **Mnożenie przez współczynniki jądra:**

   ```
   vpmullw ymm0, ymm0, ymm3 ; p[x-1] * K0
   vpmullw ymm2, ymm2, ymm3 ; p[x+1] * K0
   vpmullw ymm1, ymm1, ymm4 ; p[x] * K1
   ```

4. **Sumowanie i normalizacja Q7:**

   ```
   vpaddw ymm0, ymm0, ymm1 ; (p[x-1]*K0) + (p[x]*K1)
   vpaddw ymm0, ymm0, ymm2 ; + (p[x+1]*K0)
   vpsrlw ymm0, ymm0, 7 ; Dzielenie przez 128 (Q7 * Q7 = Q14 > Q7)
   ```

5. **Pakowanie 16-bit → 8-bit i zapis:**

   ```
   vextracti128 xmm5, ymm0, 1 ; Ekstrakcja górnej połowy YMM
   vpackuswb xmm0, xmm0, xmm5 ; Pakowanie z nasyceniem (saturated pack)
   movdqu xmmword ptr [rdi+rax+3h], xmm0
   ```

Obsługa pikseli brzegowych:

- **Pierwszy piksel (x=0):** $p'_0 = p_0 \cdot K_1 + p_1 \cdot K_2 = p_0 \cdot 0.75 + p_1 \cdot 0.25$
- **Ostatni piksel (x=width-1):** $p'_n = p_{n-1} \cdot K_2 + p_n \cdot K_1$

Faza 2: Konwolucja pionowa

Algorytm pionowy wykorzystuje analogiczną strukturę AVX2, ale operuje na pikselach z trzech kolejnych wierszy (y-1, y, y+1):

```
vpmovzxbw ymm0, xmmword ptr [rsi+rax] ; wiersz y-1
add rax, r9 ; rax += stride
vpmovzxbw ymm1, xmmword ptr [rsi+rax] ; wiersz y
add rax, r9
vpmovzxbw ymm2, xmmword ptr [rsi+rax] ; wiersz y+1

vpmullw ymm0, ymm0, ymm3 ; row[y-1] * K0
vpmullw ymm2, ymm2, ymm3 ; row[y+1] * K0
vpmullw ymm1, ymm1, ymm4 ; row[y] * K1

vpaddw ymm0, ymm0, ymm1
vpaddw ymm0, ymm0, ymm2
vpsraw ymm0, ymm0, 7 ; Normalizacja Q14 > Q7
```

Obsługa wierszy brzegowych:

- **Pierwszy wiersz (y=0):** Wykorzystuje tylko dwa wiersze (y=0, y=1) z wagami $[K_1, K_2]$

- **Ostatni wiersz (y=height-1):** Wykorzystuje wiersze (y=height-2, y=height-1) z wagami $[K_2, K_1]$

Działanie biblioteki w języku C++

Implementacja C++ wykonuje identyczne operacje co wersja asemblerowa, ale wykorzystuje zwykłe pętle skalarne (bez instrukcji SIMD):

GaussianBlurCPP_Horizontal.cpp – przykład przetwarzania środkowego piksela:

```
for (int ch = 0; ch < BYTES_PER_PIXEL; ++ch) {
uint32_t pix0 = input[pixelOffset - BYTES_PER_PIXEL + ch];
uint32_t pix1 = input[pixelOffset + ch];
uint32_t pix2 = input[pixelOffset + BYTES_PER_PIXEL + ch];
uint32_t total = (pix0 * K0) + (pix1 * K1) + (pix2 * K0);
total = total >> Q7_SHIFT;  // Dzielenie przez 128
output[pixelOffset + ch] = static_cast<unsigned char>(

   total > 255 ? 255 : total

);}
```

GaussianBlurCPP_Vertical.cpp – przetwarzanie pionowe:

```
for (int ch = 0; ch < BYTES_PER_PIXEL; ++ch) {
uint32_t pix0 = input[pixelOffset - stride + ch]; // y-1
uint32_t pix1 = input[pixelOffset + ch]; // y
uint32_t pix2 = input[pixelOffset + stride + ch]; // y+1
uint32_t total = (pix0 * K0) + (pix1 * K1) + (pix2 * K0);
total = total >> Q7_SHIFT;
output[pixelOffset + ch] = static_cast<unsigned char>(
   total > 255 ? 255 : total
);}
```

Implementacja GUI – aplikacja C# WinForms

Aplikacja graficzna (Form1.cs) zapewnia:

- **Wczytywanie obrazów** – obsługa formatów BMP, JPG, PNG z automatyczną konwersją do formatu 24-bitowego RGB
- **Wybór implementacji** – RadioButton dla wyboru między biblioteką ASM x64 a C++ Q7
- **Konfigurację wielowątkowości** – NumericUpDown (1–64 wątki) z domyślną wartością równą liczbie wątkow procesora
- **Podział wielowątkowy** – automatyczny podział obrazu na zakresy wierszy dla każdego wątku z uwzględnieniem nakładania się dla konwolucji pionowej
- **Pomiar czasu wykonania** – Stopwatch z dokładnością do setnych części milisekundy
- **Wizualizację wyników** – dwa PictureBox (obraz wejściowy i wyjściowy)
- **Zapis wyników** – SaveFileDialog z wyborem formatu (PNG/JPEG/BMP)

Wielowątkowość – strategia podziału pracy

Faza pozioma (horizontal):

```
private int CalculateStartY_Horizontal(int threadIndex, int height, int numThreads) {
int baseSize = height / numThreads;
```

```
int remainder = height % numThreads;
int startY = 0;
for (int i = 0; i < threadIndex; ++i) {
int size = baseSize + (i < remainder ? 1 : 0);
startY += size;
}
return startY;
}
```

Każdy wątek przetwarza `baseSize + (threadIndex < remainder ? 1 : 0)` wierszy bez nakładania się.

Faza pionowa (vertical) – z nakładaniem:

```
private int CalculateStartY_Vertical(int threadIndex, int height, int numThreads) {
if (threadIndex == 0)
return 0;
int prevThreadEndY = CalculateEndY_Horizontal(threadIndex - 1, height, numThreads);
return prevThreadEndY - 1; // Nakładanie o 1 wiersz
}
```

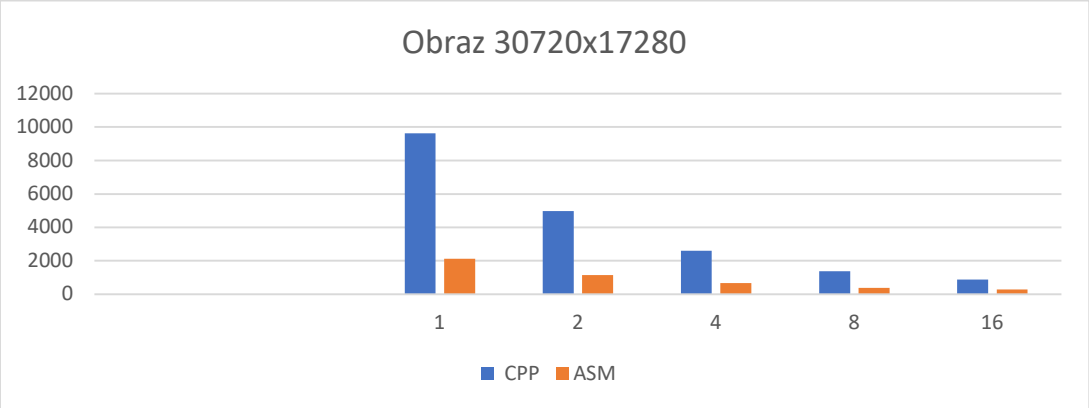
Wątki (poza pierwszym) zaczynają od `endY - 1` poprzedniego wątku, aby zapewnić dostęp do pikseli z wiersza y-1 przy konwolucji pionowej.

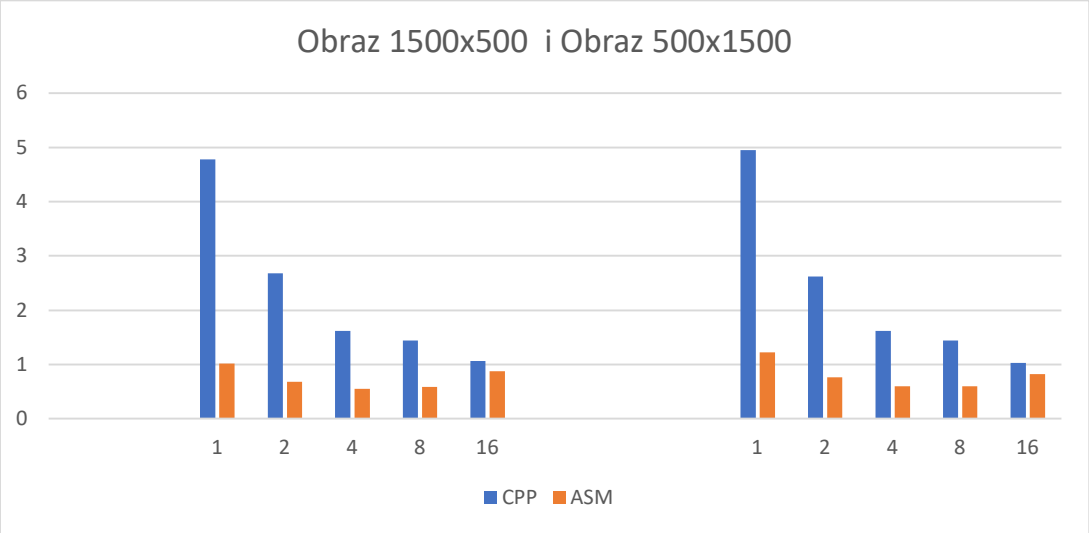
Testowanie i wyniki pomiarów

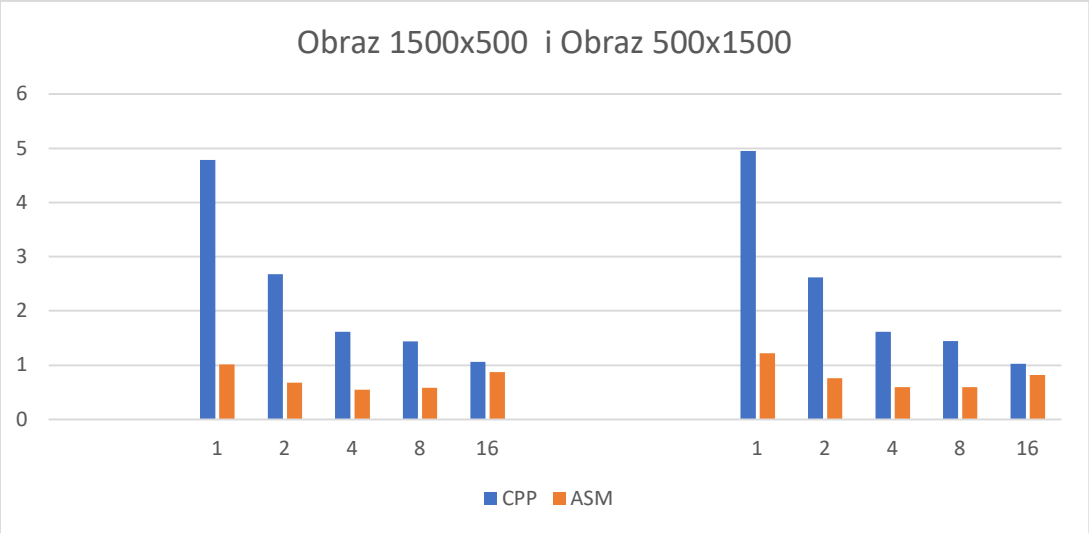
Program został przetestowany na różnych rozdzielczościach obrazów:

- **30720 x 17280 px** (bardzo duży obraz)
- **15000 x 5000 px** (duży obraz w orientacji poziomej)
- **5000 x 15000 px** (duży obraz w orientacji pionowej)
- **1500 x 500 px** (sredni obraz w orientacji pionowej)
- **500 x 1500 px (**sredni obraz w orientacji pionowej**)**

Testy wykonano na procesorze z 16 wątkami sprzętowymi.




Obraz 1500x500 i Obraz 500x1500
6
5
4
3
2
1
0
1
2
4
8
16
1
2
4
8
16
CPP
ASM


Obraz 1500x500 i Obraz 500x1500
6
5
4
3
2
1
0
1
2
4
8
16
1
2
4
8
16
CPP
ASM

Obraz 30720x17280 CPP

Ilość Wątków	1	2	4	8	16
Wynik1	9592.56	4945	2592.07	1395.79	860.33
Wynik2	9635.85	4964.5	2606.78	1355.6	905.44
Wynik3	9648.88	4980.52	2590.55	1353.59	873.22
Średnia	9625.763	4963.34	2596.467	1368.327	879.6633

Obraz 30720x17280 ASM

Ilość Wątków	1	2	4	8	16
Wynik1	2100.51	1123.5	684.47	383.88	282.62
Wynik2	2150.76	1211.13	620.4	378.05	282.72
Wynik3	2111.09	1129.69	706.43	381.53	304.83
Średnia	2120.787	1154.773	670.4333	381.1533	290.0567

Obraz 15000x5000 CPP

Ilość Wątków	1	2	4	8	16
Wynik1	1266.72	652.04	340.55	183.19	109.61
Wynik2	1396.13	649.97	338.67	178.94	110.32
Wynik3	1294.06	650.73	341.4	185.44	111.24
Średnia	1318.97	650.9133	340.2067	182.5233	110.39

Obraz 15000x5000 ASM

Ilość Wątków	1	2	4	8	16
Wynik1	303.2	169.47	88.5	57.95	40.98
Wynik2	319.95	165.59	88.95	58.33	41.58
Wynik3	301.04	156.84	87.67	59.97	42.07
Średnia	308.0633	163.9667	88.37333	58.75	41.54333

Obraz 5000x15000 CPP

Ilość Wątków	1	2	4	8	16
Wynik1	1359.96	700.17	361.91	199.57	120.31
Wynik2	1393.79	709.24	361.44	199.93	119.52
Wynik3	1368.2	705.63	363.56	198.7	120.41
Średnia	1373.983	705.0133	362.3033	199.4	120.08

Obraz 5000x15000 ASM

Ilość Wątków	1	2	4	8	16
Wynik1	294.29	159.09	90.85	58.21	42.53
Wynik2	301.16	166.08	90.41	58.48	40.87
Wynik3	344.37	162.96	90.44	60.05	40
Średnia	313.2733	162.71	90.56667	58.91333	41.13333

Obraz 1500x500 CPP

Ilość Wątków	1	2	4	8	16
Wynik1	4.85	2.8	1.68	1.41	1.03
Wynik2	4.62	2.65	1.64	1.42	1.13
Wynik3	4.88	2.59	1.52	1.48	1.02
Średnia	4.783333	2.68	1.613333	1.436667	1.06

Obraz 1500x500 ASM

Ilość Wątków	1	2	4	8	16
Wynik1	1.03	0.71	0.54	0.58	0.74
Wynik2	0.97	0.69	0.57	0.57	0.91
Wynik3	1.05	0.63	0.54	0.6	0.97
Średnia	1.016667	0.676667	0.55	0.583333	0.873333

Obraz 500x1500 CPP

Ilość Wątków	1	2	4	8	16
Wynik1	5.1	2.63	1.66	1.39	1
Wynik2	4.91	2.67	1.67	1.46	1.11
Wynik3	4.84	2.55	1.51	1.47	0.97
Średnia	4.95	2.616667	1.613333	1.44	1.026667

Obraz 500x1500 ASM

Ilość Wątków	1	2	4	8	16
Wynik1	1.21	0.75	0.6	0.62	0.86
Wynik2	1.24	0.74	0.53	0.62	0.76
Wynik3	1.21	0.79	0.65	0.55	0.84
Średnia	1.22	0.76	0.593333	0.596667	0.82

Obserwacje

1. **Przewaga wersji ASM:** Implementacja asemblerowa z AVX2 osiąga znacząco lepsze czasy wykonania niż wersja C++ dla wszystkich testowanych rozdzielczości, zwlaszcza duzych obrazow.
2. **Skalowanie wielowątkowe:** Program wykazuje poprawne skalowanie wydajności przy zwiększaniu liczby wątków do wartości równej liczbie wątków sprzętowych procesora (16). Aczkolwiek wraz ze zmniejszaniem obrazow dodatkowe watki coraz mniej przyspieszaja prace programu, w szczegolnych przypadkach spowalniajac go.
3. **Nieliniowe skalowanie ASM:** Skalowanie wydajności wersji asemblerowej względem liczby wątków nie jest idealnie proporcjonalne, co może wynikać z:
    - Narzutów synchronizacji wątków
    - Obsługi pikseli brzegowych (pierwszy/ostatni piksel/wiersz)
    - Cache contention przy dostępie do sąsiadujących obszarów pamięci przez różne wątki
4. **Efektywność wektoryzacji:** Wersja AVX2 przetwarza ~16 bajtów (około 5 pikseli RGB) na jedną iterację pętli głównej, co zapewnia znaczące przyspieszenie w porównaniu do skalarnej wersji C++ przetwarzającej po 1 pikselu.

Uruchamianie programu

Program można uruchomić na dwa sposoby:

1. **Visual Studio** – otwarcie projektu .sln i uruchomienie przez F5
2. **Executable** – uruchomienie pliku GaussianBlurUI.exe z katalogu /bin/Debug lub /bin/Release

Wymagane pliki DLL (muszą znajdować się w tym samym katalogu co .exe):

- GaussianBlurASM_Horizontal.dll
- GaussianBlurASM_Vertical.dll
- GaussianBlurCPP_Horizontal.dll
- GaussianBlurCPP_Vertical.dll

Instrukcja użytkowania:

1. Kliknij przycisk „Wczytaj obraz" i wybierz plik graficzny (BMP/JPG/PNG)
2. Wybierz implementację (ASM x64 lub C++ Q7)
3. Ustaw liczbę wątków roboczych (domyślnie: liczba rdzeni CPU)
4. Kliknij „Przetwórz" aby uruchomić filtr Gaussa
5. Po zakończeniu obliczenia przetworzony obraz pojawi się w prawym oknie
6. Opcjonalnie zapisz wynik przyciskiem „Zapisz wynik"

Wnioski

1. **Realizacja założeń:** Program działa zgodnie z założeniem projektu – poprawnie implementuje separowalny filtr Gaussa 3×3 z wykorzystaniem arytmetyki Q7 fixed-point.
2. **Wydajność AVX2:** Implementacja asemblerowa wykorzystująca instrukcje wektorowe AVX2 osiąga znacząco lepsze wyniki czasowe niż skalarna implementacja C++. Przyspieszenie wynika z przetwarzania wielu pikseli równolegle (SIMD).
3. **Skalowanie wielowątkowe:** System wielowątkowy działa poprawnie, zapewniając przyspieszenie proporcjonalne do liczby wykorzystanych wątków, aż do osiągnięcia limitu sprzętowego procesora. Nakładanie zakresów w fazie pionowej zostało poprawnie zaimplementowane.
4. **Ograniczenia implementacji:** Skalowanie wydajności wersji asemblerowej nie jest idealnie liniowe względem liczby wątków, co może wynikać z narzutów synchronizacji, obsługi przypadków brzegowych w kodzie skalarnym oraz dostępu do pamięci. Potencjalne ulepszenia:
   - Wektoryzacja obsługi pikseli/wierszy brzegowych
   - Optymalizacja dostępu do pamięci (cache-friendly memory access patterns)
   - Eksperymentowanie z rozmiarem bloków przetwarzanych przez każdy wątek
5. **Jakość artefaktów wizualnych:** Arytmetyka Q7 fixed-point zapewnia wystarczającą precyzję dla tego typu filtru, zachowując szybkość operacji całkowitoliczbowych.
6. **Praktyczna użyteczność:** Aplikacja GUI upraszcza testowanie i prezentację działania algorytmu, umożliwiając łatwe porównanie wydajności obu implementacji oraz eksperymentowanie z różnymi konfiguracjami wielowątkowości.

Ostateczna ocena: Projekt z powodzeniem demonstruje przewagę niskopoziomowej optymalizacji asemblerowej z instrukcjami SIMD nad wysokopoziomową implementacją C++ dla intensywnych obliczeniowo zadań przetwarzania obrazów. Wciąż istnieje przestrzeń do dalszej optymalizacji, szczególnie w kontekście eliminacji operacji skalarnych i poprawy wykorzystania cache procesora.